

Base de datos SEPA minorista

Documentación técnica para su procesamiento

Instituto de Economía — INECO / UADE

17 de junio de 2026

Resumen

Este documento describe en detalle cómo está constituida la base de datos del Sistema Electrónico de Publicidad de Precios Argentinos (SEPA) en su variante de archivos consolidados con granularidad diaria, cómo se identifican los supermercados y sus sucursales, qué información contiene cada archivo, cómo se leen los precios y los códigos de producto, cómo se clasifican las sucursales por cadena comercial y por ubicación geográfica (provincias, departamentos y localidades mediante latitud y longitud), y qué errores y consideraciones hay que tener en cuenta para procesarla correctamente. Está dirigido a economistas y estudiantes de economía que vayan a trabajar con la base sin conocimiento previo de su estructura interna.

Índice

1. Introducción y alcance	3
2. Origen y estructura general de los archivos	3
3. Estructura de los archivos de precios	4
3.1. Columnas de identificación	4
3.2. Columnas de precio	4
3.3. Valores faltantes y tipo de dato	5
4. Identificación de comercios y sucursales	5
5. Clasificación por cadena comercial	5
5.1. Grupo corporativo frente a banner comercial	5
5.2. Diccionario de mapeo	5
5.3. Comercios regionales sin nombre asignado	6
5.4. Implementación vectorizada	6
6. Los archivos maestros de referencia	7
6.1. Maestro de provincias	7
6.2. Maestro de sucursales	8
6.3. Maestro de productos	9
7. Códigos de producto (EAN / GTIN) y su normalización	10
7.1. Qué es el id_producto	10
7.2. Leer siempre como texto	10
7.3. Normalización para la unión	10
7.4. Códigos PLU de balanza	10
8. Lectura e interpretación de los precios	11
8.1. El factor de escala: centavos o pesos	11
8.2. Agregación diaria	11
9. Georreferenciación: provincia, departamento y localidad por latitud y longitud	12
9.1. Coordenadas en el maestro	12
9.2. Provincia: unir por código ISO, no por texto	12
9.3. Reclasificación por coordenadas	12
9.4. Barrio de CABA por coordenadas	13
10. Errores y consideraciones a tener en cuenta	13
11. Recomendaciones de procesamiento	15
11.1. Modelo de datos en estrella	15
11.2. Lectura que no agota la memoria	15
11.3. Agregar temprano y cachear	16
11.4. Orden de las uniones	16

1. Introducción y alcance

Más de 2.300 sucursales, pertenecientes a 16 cadenas comerciales distintas (Carrefour, Changomás, Disco, Coto, Jumbo. La Anonima, entre otras) están obligadas a reportar, diariamente, los precios de todos los productos que tengan disponibles en góndola. Esta información es consolidada en la base SEPA, en donde se reúnen más de 72.000 productos únicos y sus precios, a partir de entre 13 y 15 millones de registros diarios (combinaciones producto-sucursal).

El SEPA es un registro administrativo del Estado argentino que obliga a las principales cadenas de supermercados a publicar sus precios de lista. Los datos son públicos y se descargan desde el portal oficial de datos abiertos (datos.produccion.gob.ar/dataset/sepa-precios). La base es masiva: cada semestre reúne del orden de cincuenta millones de filas a nivel de producto por sucursal, cubriendo decenas de miles de productos en miles de sucursales de todo el país.

Existen dos presentaciones distintas de los mismos datos. La primera es un conjunto de archivos por cadena y por día, separados por barra vertical, con tres archivos por cadena (comercio, sucursales y productos). La segunda, que es la que documenta este informe, es la presentación **consolidada nacional**: archivos mensuales que traen, en formato ancho, una columna de precio por cada día del mes, agrupados a su vez en archivos comprimidos semestrales. Esta segunda presentación es la más conveniente para construir series temporales largas y es la que se asume a lo largo de todo el documento.

En este documento se detalla cómo se nombran y organizan los archivos, qué columnas contiene cada uno, cómo se identifican comercios y sucursales, cómo se traduce esa identificación a nombres de cadena comercial reconocibles, qué traen los tres archivos maestros de referencia, cómo se interpretan los códigos de barras de los productos, cómo se leen e interpretan los precios, cómo se georreferencian las sucursales, qué errores y suciedades concretas tiene la base, y qué procesamiento conviene adoptar.

2. Origen y estructura general de los archivos

La base se distribuye en archivos comprimidos semestrales con el patrón de nombre **AAAAS.zip**, donde **AAAA** es el año y **S** es la mitad del año (**A** para el primer semestre, enero a junio, y **B** para el segundo, julio a diciembre). Por ejemplo, **2026A.zip** contiene los datos de enero a junio de 2026, y se va completando mes a mes a medida que avanza el semestre.

Dentro de cada ZIP, los datos vienen en archivos mensuales partidos en dos mitades de mes, comprimidos individualmente con gzip. El patrón de nombre es:

MMAAAA_pais_parteNCOMPLETO.csv.gz

donde **MM** es el mes con dos dígitos, **AAAA** es el año, y **N** es el número de parte. La parte 1 cubre aproximadamente los días 1 a 15 del mes y la parte 2 el resto del mes. Así, **012026_pais_parte1COMPLETO.csv.gz** corresponde a la primera quincena de enero de 2026 y **012026_pais_parte2COMPLETO.csv.gz** a la segunda. El sufijo **pais** indica que es la consolidación nacional, y **COMPLETO** que el archivo trae el universo completo de productos y sucursales del período, no una muestra.

Cada `.csv.gz` es un único archivo de texto separado por comas, codificado en UTF-8 y comprimido con gzip. La descompresión de cada parte puede superar holgadamente el gigabyte en memoria, por lo que nunca debe cargarse de golpe: en la Sección 11 se detalla la lectura por bloques.

3. Estructura de los archivos de precios

Los archivos de precios son, conceptualmente, la **tabla de hechos** de la base: filas identificadas por claves, con los valores numéricos (precios) como medida. No contienen la descripción del producto ni el nombre de la sucursal; esos atributos se resuelven uniendo contra los archivos maestros (Sección 6). Esta separación entre hechos y dimensiones es deliberada y es la clave para procesar la base con eficiencia.

3.1. Columnas de identificación

Cada fila se identifica por cinco campos, todos los cuales deben leerse como texto (no como número):

Campo	Significado
<code>id_comercio</code>	Identificador del comercio (empresa) asignado por la Secretaría de Comercio.
<code>id_bandera</code>	Identificador de la bandera o unidad de negocio dentro del comercio.
<code>id_sucursal</code>	Identificador interno y estable de la sucursal dentro del comercio.
<code>sucursales_provincia</code>	Código de provincia según la norma ISO 3166-2 (por ejemplo AR-C para CABA, AR-B para Buenos Aires).
<code>id_producto</code>	Código del producto: un EAN/GTIN o un código interno del comercio de trece dígitos.

La identidad completa de una sucursal física es la terna (`id_comercio`, `id_bandera`, `id_sucursal`): ninguno de los tres campos es único por sí solo. La identidad de una observación de precio es esa terna más `id_producto` más el día.

3.2. Columnas de precio

El resto de las columnas son los precios, una columna por cada día del período de la parte, con el patrón de nombre `precio_AAAAMMDD`. Por ejemplo, la parte 1 de enero de 2026 tendrá columnas `precio_20260101`, `precio_20260102`, hasta `precio_20260115`. Cada producto en cada sucursal ocupa una sola fila, y su evolución de precio a lo largo de los días se lee horizontalmente.

El valor de cada celda de precio es el precio de lista del producto en esa sucursal ese día. El precio de lista es el precio al que cualquier consumidor puede adquirir el producto sin descuentos, sin tarjeta de fidelidad y sin segmentación (jubilado, estudiante, etcétera). No incluye promociones

ni precios de segunda unidad. Esta distinción es importante para la interpretación económica: el SEPA mide precios de lista, no precios efectivamente pagados.

3.3. Valores faltantes y tipo de dato

Cuando un producto no tiene precio reportado un día determinado, la celda contiene el texto literal NA, no un valor nulo. Es imprescindible reemplazar ese texto por un faltante real antes de cualquier operación numérica, de lo contrario toda la columna se interpreta como texto. Las columnas de precio deben convertirse a punto flotante; conviene usar `float32` para reducir a la mitad el consumo de memoria respecto del `float64` por defecto.

4. Identificación de comercios y sucursales

Cada sucursal es determinada por la combinación (`id_comercio`, `id_bandera`, `id_sucursal`). El `id_comercio` identifica a la empresa; el `id_bandera` distingue las distintas banderas o formatos comerciales dentro de esa empresa; el `id_sucursal` es el código interno del local. Un mismo `id_sucursal` puede repetirse entre comercios distintos, por lo que jamás debe usarse como clave única por sí solo.

Un error frecuente al explorar la base es contar los valores únicos de `id_bandera` y concluir que hay solo cinco o seis cadenas. Esto es incorrecto. El `id_bandera` no identifica al banner comercial (el nombre que ve el consumidor en la puerta del local), sino a la unidad de negocio dentro del comercio. Una misma empresa opera varios banners con distinto `id_bandera`: por ejemplo, el grupo Cencosud opera Veja, Disco y Jumbo, que comparten `id_comercio` pero tienen `id_bandera` distinto. Para recuperar el nombre comercial real hay que combinar `id_comercio` con `id_bandera`, como se explica en la Sección 5.

5. Clasificación por cadena comercial

5.1. Grupo corporativo frente a banner comercial

Conviene distinguir dos niveles. El **grupo corporativo** es la empresa propietaria (Cencosud, Carrefour, etcétera) y se corresponde con el `id_comercio`. El **banner comercial** es el nombre de fantasía bajo el que opera cada formato de local (Veja, Disco, Jumbo, Carrefour Market, Carrefour Express) y se obtiene combinando `id_comercio` con `id_bandera`. Para análisis de competencia y dispersión de precios suele interesar el banner; para medir cobertura nacional suele interesar el grupo.

5.2. Diccionario de mapeo

La traducción de la combinación de identificadores a nombre de cadena se hace con dos diccionarios. El primero, para banners que requieren la combinación de comercio y bandera:

id_comercio	id_bandera	Banner comercial
9	1	Vea
9	2	Disco
9	3	Jumbo
10	1	Carrefour
10	2	Carrefour Market
10	3	Carrefour Express
11	2	ChangoMas
11	4	Hiper ChangoMas
11	5	Mi ChangoMas
16	1	Hipermercado Libertad
16	2	Mini Libertad

El segundo, para comercios que tienen una sola bandera relevante y pueden mapearse solo por `id_comercio`:

id_comercio	Cadena
2	La Anónima
12	Coto
13	Cooperativa Obrera
15	DIA

5.3. Comercios regionales sin nombre asignado

Además de los grupos principales, la base incluye una cantidad de comercios regionales más chicos (con `id_comercio` como 19, 21, 23, 61, 63, 65, 2000, 2002, 2004, 2005, 2011, 3001, entre otros) que no están en el diccionario. La estrategia recomendada es asignarles una etiqueta genérica del tipo Comercio `<id_comercio>` para no perderlos del análisis. Algunos de estos comercios reportan precios atípicos, propios de tiendas de especialidad o productos importados, por lo que conviene revisarlos antes de incluirlos en agregados nacionales.

5.4. Implementación vectorizada

La traducción debe hacerse de forma vectorizada y nunca con `apply` fila por fila: con decenas de millones de filas, un `apply` en Python ejecuta millones de llamadas y agota la memoria. El patrón correcto construye una clave compuesta y la mapea contra un diccionario:

```

1 CADENAS_COMPUESTAS = {
2     ('9','1'):'Vea', ('9','2'):'Disco', ('9','3'):'Jumbo',
3     ('10','1'):'Carrefour', ('10','2'):'Carrefour Market',
4     ('10','3'):'Carrefour Express',
5     ('11','2'):'ChangoMas', ('11','4'):'Hiper ChangoMas',
6     ('11','5'):'Mi ChangoMas',
7     ('16','1'):'Hipermercado Libertad', ('16','2'):'Mini Libertad',
8 }
9 CADENAS_SIMPLES = {'2':'La Anonima', '12':'Coto',
10                    '13':'Cooperativa Obrera', '15':'DIA'}
11
12 # 1) clave compuesta id_comercio_id_bandera
13 lookup_comp = {f"{c}_{b}": v for (c,b), v in CADENAS_COMPUESTAS.items()}
14 clave = df['id_comercio'] + '_' + df['id_bandera']
15 df['nombre_cadena'] = clave.map(lookup_comp)
16
17 # 2) fallback por comercio simple
18 falta = df['nombre_cadena'].isna()
19 df.loc[falta, 'nombre_cadena'] = df.loc[falta, 'id_comercio'].map(CADENAS_SIMPLES)
20
21 # 3) residuo: etiqueta generica
22 falta = df['nombre_cadena'].isna()
23 df.loc[falta, 'nombre_cadena'] = 'Comercio ' + df.loc[falta, 'id_comercio']

```

Listing 1: Asignacion de nombre de cadena, vectorizada

6. Los archivos maestros de referencia

Los archivos de precios traen solo identificadores. Toda la información descriptiva (nombre y tipo de la sucursal, coordenadas, descripción y categoría del producto, nombre legible de la provincia) vive en tres archivos maestros que se unen contra la tabla de precios. Funcionan como las **dimensiones** del modelo.

6.1. Maestro de provincias

Es el más chico y el más limpio: veinticuatro filas, una por jurisdicción. Mapea el código ISO de la columna `sucursales_provincia` al nombre legible de la provincia.

Columna	Contenido
<code>sucursales_provincia</code>	Código ISO 3166-2 (AR-C, AR-B, AR-K, ...).
<code>provincia</code>	Nombre legible y canónico (CABA, Buenos Aires, Catamarca, ...).

Este maestro es la pieza clave para georreferenciar de forma confiable: como se verá en la Sección 9,

unir por el código ISO evita por completo los problemas de suciedad del nombre textual de la provincia.

6.2. Maestro de sucursales

Reúne alrededor de tres mil seiscientas sucursales con sus atributos completos. La clave de unión con la tabla de precios es la terna (`id_comercio`, `id_bandera`, `id_sucursal`).

Columna	Contenido
<code>id_comercio</code>	Identificador del comercio.
<code>id_bandera</code>	Identificador de la bandera.
<code>id_sucursal</code>	Identificador de la sucursal.
<code>sucursales_nombre</code>	Nombre con que se reconoce la sucursal.
<code>sucursales_tipo</code>	Tipo de local según líneas de caja (ver advertencia más abajo).
<code>sucursales_calle</code>	Calle de la puerta principal, normalizada (sin número).
<code>sucursales_numero</code>	Numeración de la puerta principal.
<code>sucursales_latitud</code>	Latitud en WGS84, seis decimales, punto decimal.
<code>sucursales_longitud</code>	Longitud en WGS84, seis decimales, punto decimal.
<code>sucursales_observaciones</code>	Texto libre de localización.
<code>sucursales_barrio</code>	Barrio o subdivisión de la localidad.
<code>sucursales_codigo_postal</code>	Código postal numérico.
<code>sucursales_localidad</code>	Localidad.
<code>sucursales_provincia</code>	Código ISO de provincia (a unir con el maestro de provincias).
<code>sucursales_*_horario_atencion</code>	Siete columnas, una por día de la semana, con el horario.
PROVINCIA	Nombre textual de la provincia (campo agregado, con suciedad).
REGION	Región geográfica agregada (AMBA, Pampeana, Patagonia, Noroeste, Cuyo, Noreste).

La distribución de sucursales por región del maestro es, aproximadamente: AMBA con poco más de 2.100 locales, Pampeana con cerca de 800, Patagonia con 300, Noroeste con 165, Cuyo con 153, y Noreste con 113. Esta fuerte concentración en AMBA debe tenerse presente al construir promedios nacionales: conviene ponderar por población provincial en lugar de promediar sucursales sin peso.

6.3. Maestro de productos

Es el más grande: alrededor de 166.700 productos. La clave de unión con la tabla de precios es `producto_sepa_id`, que se corresponde con `id_producto` de los archivos de precios. Organiza cada producto en tres niveles jerárquicos: rubro, categoría y subcategoría.

Columna	Contenido
<code>producto_sepa_id</code>	Código del producto (EAN/GTIN), clave de unión con <code>id_producto</code> .
<code>producto_ean</code>	Indica si el código es un EAN/GTIN estándar (1) o un código interno (0).
<code>producto_descripcion</code>	Descripción completa: nombre, marca, presentación, cantidad.
<code>producto_marca</code>	Marca comercial.
<code>producto_cantidad_presentacion</code>	Cantidad de producto en la presentación de venta.
<code>producto_unidad_medida_presentacion</code>	Unidad de medida de la presentación (Gr, Ml, Kg, Lt, ...).
<code>producto_cantidad_referencia</code>	Cantidad en la unidad de referencia.
<code>producto_unidad_medida_referencia</code>	Unidad de referencia (para el precio por kilo o por litro).
<code>rubro</code>	Primer nivel de clasificación (Almacén, Frescos, Bebidas, ...).
<code>categoria</code>	Segundo nivel (Lácteos, Fiambrería, Conservas, Infusiones, ...).
<code>subcategoria</code>	Tercer nivel, más fino (Quesos Semiduros, Patés y Picadillos, ...).
<code>proveedor</code>	Razón social del proveedor.
<code>cuit</code>	CUIT del proveedor.
<code>producto_blacklist</code>	Marca de producto excluido.
<code>producto_nuevo</code>	Marca de producto nuevo.
<code>producto_imagen</code>	Enlace a imagen, cuando existe.

El maestro distingue, en el universo completo, del orden de 194 categorías y 479 subcategorías. Conviene saber, además, que el valor de la columna `categoria` es una etiqueta literal del maestro (por ejemplo, los lácteos figuran con `categoria = Lácteos`, no con palabras como “leche” o “queso”). Cualquier filtro por categoría debe coincidir con esa etiqueta literal, no con el nombre genérico del producto.

7. Códigos de producto (EAN / GTIN) y su normalización

7.1. Qué es el `id_producto`

El `id_producto` es, en la mayoría de los casos, el código de barras del producto bajo el estándar EAN/GTIN. Puede ser un GTIN-13 (el habitual de góndola, trece dígitos), un GTIN-8 (productos chicos), un GTIN-12 o UPC-A (doce dígitos, típico de productos de origen estadounidense), o, cuando el comercio no dispone de un código de barras, un código interno propio de trece dígitos. La columna `producto_ean` indica con un 1 que el código es un estándar y con un 0 que es un código interno del comercio.

7.2. Leer siempre como texto

La regla más importante de toda la base: el `id_producto` debe leerse **siempre como cadena de texto**, nunca como número entero. Muchos códigos tienen ceros iniciales significativos; si en cualquier paso del procesamiento el campo se convierte a entero, esos ceros se pierden y el código deja de coincidir con el maestro. La forma de fijar el tipo al leer es pasar `dtype={'id_producto':str}` a la función de lectura.

7.3. Normalización para la unión

Para unir la tabla de precios con el maestro conviene normalizar el código de la misma manera en ambos lados. La práctica habitual es quitar los ceros iniciales (`lstrip`) para construir una clave de unión robusta, y por separado rellenar con ceros a la izquierda hasta trece dígitos (`zfill(13)`) solo al momento de exportar a planilla, para preservar la forma EAN-13 legible.

```
1 # clave robusta para el merge (ambos lados igual)
2 df['ean_norm'] = df['id_producto'].astype(str).str.lstrip('0')
3
4 # forma EAN-13 legible solo para exportar (no pierde ceros)
5 df['ean_13'] = df['id_producto'].astype(str).str.zfill(13)
```

Listing 2: Normalización de códigos de producto

7.4. Códigos PLU de balanza

Los productos vendidos por peso en góndola (frutas, verduras, fiambres cortados al momento) llevan códigos PLU generados por las balanzas del local. Estos códigos empiezan con los prefijos 27, 28 o 29 y son efímeros: el mismo producto puede tener códigos distintos en distintas sucursales o fechas, y no aparecen de forma consistente en la serie histórica. En consecuencia, no sirven para construir series temporales: si se incluye un código PLU en una canasta, su serie quedará vacía.

8. Lectura e interpretación de los precios

8.1. El factor de escala: centavos o pesos

La unidad en que están expresados los precios cambió a lo largo del tiempo. Hasta mediados de 2025, los precios venían expresados en **centavos** y había que dividirlos por cien. Desde el segundo semestre de 2025 en adelante, los precios ya vienen en **pesos** y no hay que dividir. Nunca debe codificarse una división fija por cien: hay que autodetectar el factor. El método adecuado es mirar la mediana global de los precios crudos: si supera diez mil, los datos están en centavos y se divide por cien; si no, están en pesos y se deja el factor en uno. Esto funciona porque la mediana de un precio de góndola en pesos ronda unos pocos miles, muy lejos del rango que tomaría expresado en centavos.

```
1 mediana = df['precio_promedio'].median()
2 FACTOR = 100 if mediana > 10_000 else 1
3 if FACTOR == 100:
4     df['precio_promedio'] = (df['precio_promedio'] / 100).astype('float32')
```

Listing 3: Autodetección del factor de escala

Un método alternativo, aún más robusto, usa un puñado de productos de referencia de precio conocido (por ejemplo sal fina, fideos secos, lavandina) y decide el factor según el rango en que cae su mediana.

8.2. Agregación diaria

Como cada parte trae una columna por día, lo primero es separar las columnas de precio del resto. A partir de ahí hay dos caminos. Si interesa el detalle diario, se puede pasar a formato largo (una fila por producto, sucursal y día). Si interesa el resumen del período, se calcula por fila el promedio de las columnas de precio y la cantidad de días con precio reportado, sin materializar el formato largo (mucho más liviano en memoria). El conteo de días con precio sirve además como medida de cobertura temporal del producto en esa sucursal.

```
1 import numpy as np
2
3 price_cols = [c for c in chunk.columns if c.startswith('precio_')]
4 precios = chunk[price_cols].replace('NA', np.nan).astype('float32')
5 chunk = chunk.drop(columns=price_cols)
6 chunk['precio_promedio'] = precios.mean(axis=1).astype('float32')
7 chunk['dias_conPrecio'] = precios.notna().sum(axis=1).astype('int16')
```

Listing 4: Resumen por fila sin explotar a formato largo

9. Georreferenciación: provincia, departamento y localidad por latitud y longitud

9.1. Coordenadas en el maestro

El maestro de sucursales trae `sucursales_latitud` y `sucursales_longitud` en el sistema de referencia WGS84, con seis decimales y punto como separador decimal. Vienen como texto, así que hay que convertirlas a punto flotante. Conviene validar el rango: para Argentina continental la latitud cae aproximadamente entre -55 y -21 , y la longitud entre -74 y -53 . Coordenadas fuera de ese rango (o vacías, o en cero) deben marcarse como inválidas y no usarse para el mapeo.

9.2. Provincia: unir por código ISO, no por texto

La forma confiable de asignar provincia no es leer la columna textual `PROVINCIA` del maestro de sucursales (que está sucia, como se detalla en la Sección 10), sino unir el código ISO `sucursales_provincia` contra el maestro de provincias. El código ISO es estable y no tiene problemas de mayúsculas, tildes ni prefijos. Esta unión resuelve de raíz la mayoría de los problemas de normalización geográfica.

```
1 prov = pd.read_excel('maestro-provincias.xlsx') # ISO -> nombre
2 suc = suc.merge(prov, on='sucursales_provincia', how='left')
3 # suc['provincia'] queda con el nombre canonico
```

Listing 5: Provincia confiable por código ISO

9.3. Reclasificación por coordenadas

Aun con la unión por ISO, algunas sucursales pueden traer mal cargada la provincia en el origen. La regla es no descartarlas, sino **reclasificarlas por sus coordenadas**. Hay dos técnicas. La rápida y aproximada usa un rectángulo (bounding box) por provincia: si el punto cae dentro del rectángulo de una provincia, se le asigna esa provincia. La precisa usa los polígonos reales de las provincias (un archivo GeoJSON de las veinticuatro jurisdicciones) y resuelve el punto-en-polígono.

A modo de ejemplo, algunos rectángulos aproximados (latitud mínima, latitud máxima, longitud mínima, longitud máxima) son: CABA (-34.72 , -34.52 , -58.54 , -58.33); Tucumán (-28.0 , -26.0 , -66.5 , -64.5); Jujuy (-24.5 , -21.5 , -67.5 , -63.5). Para las veinticuatro provincias conviene, en lugar de transcribir rectángulos a mano, usar directamente el GeoJSON oficial con polígonos, que es más exacto en los límites:

```
1 import json
2 from shapely.geometry import shape, Point
3
4 with open('ar.json', encoding='utf-8') as f:
5     geo = json.load(f)
6     poligonos = [(feat['properties']['name'], shape(feat['geometry']))
7                  for feat in geo['features']]
```

```

8
9 def provincia_por_coords(lat, lon):
10     p = Point(lon, lat)    # ojo: (x=lon, y=lat)
11     for nombre, poly in poligonos:
12         if poly.contains(p):
13             return nombre
14     return None

```

Listing 6: Reclasificación punto-en-polígono con GeoJSON de provincias

Nótese que algunos GeoJSON nombran a la Capital como “Ciudad de Buenos Aires” o “Ciudad Autónoma de Buenos Aires”; conviene mapear ese nombre a “CABA” al unir, para que coincida con el resto del análisis.

9.4. Barrio de CABA por coordenadas

Para las sucursales de la Ciudad de Buenos Aires se puede ir un nivel más fino y asignar el barrio. El método es el mismo punto-en-polígono, pero usando un GeoJSON de los cuarenta y ocho barrios de CABA (disponible en el portal de datos abiertos de la Ciudad). Para cada sucursal con coordenadas dentro de CABA, se busca en qué polígono de barrio cae su punto. La alternativa rápida con rectángulos también es posible, pero los barrios tienen límites irregulares y el rectángulo genera más errores que en el caso provincial, así que se recomienda el polígono.

```

1 with open('barrios_caba.geojson', encoding='utf-8') as f:
2     geo_b = json.load(f)
3     barrios = [(feat['properties']['BARRIO'], shape(feat['geometry']))
4                 for feat in geo_b['features']]
5
6 def barrio_por_coords(lat, lon):
7     p = Point(lon, lat)
8     for nombre, poly in barrios:
9         if poly.contains(p):
10             return nombre
11     return None

```

Listing 7: Barrio de CABA punto-en-polígono

Una opción más cómoda que manejar los polígonos a mano es usar **geopandas**: cargar las sucursales como **GeoDataFrame** de puntos y hacer un **sjoin** espacial contra el GeoJSON de provincias o de barrios. El resultado es el mismo y el código es más breve.

10. Errores y consideraciones a tener en cuenta

Esta sección reúne todas las suciedades y trampas concretas de la base. Tenerlas presentes evita errores silenciosos que no rompen el procesamiento pero arruinan los resultados.

1. **Factor de escala de precios.** Hasta mediados de 2025 los precios están en centavos y desde el segundo semestre de 2025 en pesos. Una división fija por cien produce precios cien veces más bajos en los datos recientes. Hay que autodetectar el factor (Sección 8).
2. **Faltantes como texto.** Los faltantes vienen como el texto NA, no como nulo. Sin un reemplazo explícito, toda la columna de precios queda como texto y cualquier operación numérica falla o da resultados absurdos.
3. **Códigos de producto como entero.** Si el `id_producto` se lee o se castea a entero, los ceros iniciales se pierden y el código deja de coincidir con el maestro. Leer siempre como texto.
4. **Longitud variable de los códigos.** No todos los códigos tienen trece dígitos: hay una cantidad apreciable de diecisiete, dieciocho y veinte dígitos (códigos logísticos o concatenados).
5. **Códigos PLU efímeros.** Los códigos con prefijo 27, 28 o 29 (productos pesados en balanza) no tienen historia consistente y no sirven para series temporales.
6. **Columna PROVINCIA sucia.** El nombre textual de provincia en el maestro de sucursales tiene inconsistencias: aparece “San juan” con jota minúscula, “Ciudad Autónoma de Buenos Aires” en vez de CABA, y “Provincia de Buenos Aires” con prefijo. Conviene ignorar esta columna y derivar la provincia del código ISO uniendo contra el maestro de provincias.
7. **Espacio no separable en REGION.** La columna de región trae un carácter de espacio no separable (el carácter Unicode U+00A0) al final de sus valores, de modo que “Noreste” figura en realidad como “Noreste ”. Un `strip` normal no siempre lo quita; conviene reemplazar explícitamente ese carácter por un espacio común y luego recortar.
8. **Tipo de sucursal inconsistente.** La columna `sucursales_tipo` mezcla mayúsculas y minúsculas (“Supermercado”, “SUPERMERCADO”) y categorías que no figuran en la especificación oficial (“Mayorista”, “Tienda Fisica”, “Tienda Virtual”, “Sin Clasificacion”), además de las cuatro previstas (Hipermercado, Supermercado, Autoservicio, Tradicional) y la web. Hay que normalizarla a minúsculas y mapearla a un conjunto controlado de categorías antes de usarla.
9. **Duplicación de rubros por mayúsculas y espacios.** En el maestro de productos, la columna `rubro` trae el mismo rubro escrito de varias formas (“Bazar” y “BAZAR”; “Limpieza” y “Limpieza ” con espacio final; “Climatización” y “CLIMATIZACIÓN”). Sin canonicalizar (pasar a una capitalización uniforme y recortar espacios) se cuentan rubros duplicados.
10. **id_bandera no es el banner.** Como se explicó, contar valores de `id_bandera` subestima la cantidad de cadenas. El banner comercial se obtiene combinando comercio y bandera.
11. **Comercios regionales sin nombre.** Una porción de los comercios no está en el diccionario de cadenas. No deben descartarse: se etiquetan de forma genérica y se revisan aparte por sus precios eventualmente atípicos.

12. **Precio de lista, no precio efectivo.** La base no incluye descuentos, promociones ni precios de fidelización. El precio pagado por el hogar puede ser menor.
13. **Cobertura desigual de productos.** Las carnes frescas, frutas, verduras y el pan tienen muy baja cobertura en supermercados de cadena (se venden mayormente en comercios fuera del SEPA o por peso con códigos PLU). Las canastas construidas con esta base no representan bien esos rubros.
14. **Concentración geográfica en AMBA.** Más de la mitad de las sucursales están en AMBA. Para promedios nacionales conviene ponderar por población provincial y no promediar sucursales sin peso.
15. **Variantes del mismo producto.** El mismo ítem en distintos gramajes o presentaciones tiene códigos distintos. No son errores, pero inflan el conteo de productos; si se busca un producto representativo por categoría hay que limpiar variantes.

11. Recomendaciones de procesamiento

11.1. Modelo de datos en estrella

Conviene pensar la base como un esquema en estrella: una tabla de hechos de precios y cuatro dimensiones. La tabla de hechos sale de los archivos de precios, con clave (`id_comercio`, `id_bandera`, `id_sucursal`, `id_producto`, `fecha`) y el precio como medida. Las dimensiones son producto (del maestro de productos), sucursal (del maestro de sucursales), cadena (del diccionario de banners) y provincia (del maestro de provincias). Este modelo mantiene la tabla grande lo más fina posible y deja todos los atributos descriptivos en tablas chicas que se unen solo cuando hacen falta.

11.2. Lectura que no agota la memoria

El mayor riesgo técnico es quedarse sin memoria. Cada parte mensual descomprimida puede ocupar varios gigabytes. La estrategia segura tiene cuatro elementos: descomprimir el `.gz` a disco en lugar de a memoria; leer el CSV por bloques; fijar tipos eficientes al leer (los identificadores como texto, los precios como `float32`); y reducir cada bloque de inmediato (calcular el promedio y el conteo de días por fila y descartar las columnas diarias). Así nunca se mantiene en memoria el frame ancho completo.

```
1 import zipfile, gzip, shutil, gc
2 from pathlib import Path
3 import numpy as np, pandas as pd
4
5 def cargar_parte(zip_path, nombre_csv_gz, tmp=Path('/tmp/sepa')):
6     tmp.mkdir(exist_ok=True)
7     destino = tmp / nombre_csv_gz
8     # 1) descomprimir a disco en streaming (no a memoria)
```

```

9     with zipfile.ZipFile(zip_path) as z:
10         with z.open(nombre_csv_gz) as src, open(destino, 'wb') as dst:
11             shutil.copyfileobj(src, dst, length=4*1024*1024)
12     # 2) leer por bloques, reducir cada bloque
13     piezas = []
14     with gzip.open(destino, 'rt', encoding='utf-8') as g:
15         for chunk in pd.read_csv(
16             g,
17             dtype={'id_comercio':str, 'id_bandera':str,
18                   'id_sucursal':str, 'id_producto':str,
19                   'sucursales_provincia':str},
20             chunksize=200_000, low_memory=False):
21             pc = [c for c in chunk.columns if c.startswith('precio_')]
22             precios = chunk[pc].replace('NA', np.nan).astype('float32')
23             chunk = chunk.drop(columns=pc)
24             chunk['precio_promedio'] = precios.mean(axis=1).astype('float32')
25             chunk['dias_con_precio'] = precios.notna().sum(axis=1).astype('int16')
26             piezas.append(chunk)
27         del precios
28     destino.unlink()
29     df = pd.concat(piezas, ignore_index=True)
30     del piezas; gc.collect()
31     return df

```

Listing 8: Lectura por bloques con descompresión a disco

11.3. Agregar temprano y cachear

Después de enriquecer la tabla con la geografía y el nombre de cadena, conviene agregar de inmediato al nivel de análisis (por ejemplo, producto por cadena por provincia) y liberar el frame grande con `del` y `gc.collect()`. Pasar de cincuenta millones de filas a unos pocos millones reduce la memoria de varios gigabytes a unos cientos de megabytes, y todo el análisis posterior corre sobre ese frame chico. Conviene además guardar los resultados intermedios costosos en formato Parquet (con compresión `snappy`): si el proceso se interrumpe, se retoma desde el último Parquet en lugar de reprocesar todo.

11.4. Orden de las uniones

El orden recomendado es: leer la parte de precios reducida; unir contra el maestro de sucursales por la terna de identificación para traer coordenadas, tipo y región; derivar la provincia uniéndolo al código ISO contra el maestro de provincias; reclasificar por coordenadas las sucursales con provincia inconsistente; asignar el nombre de cadena con el diccionario vectorizado; y recién entonces unir contra el maestro de productos por el código normalizado para traer rubro, categoría y descripción. Unir los maestros después de haber reducido la tabla de hechos mantiene cada unión barata.